

are typically optimised for broad, consumer-oriented applications rather than specialised needs.

For enterprises that demand domain-specific accuracy and efficiency, blindly pursuing larger parameter counts can be both costly and counterproductive. The real question isn't how big the model is, but whether its scale is right-sized for the task at hand.

Analysing LLMs through a technical lens

A generative AI model's parameter count dictates its hardware compute demands. Even a small LLM model such as a 7 billion parameter model can consume approximately around 67GB of RAM at a low precision of 16-bit representation. Scaling to larger models skyrockets memory requirements into the hundreds of gigabytes, forcing enterprises to deploy high-end GPUs, multi-socket CPUs, or specialised NPUs to sustain inference throughput.

These hardware requirements aren't just a procurement issue; they influence the entire operational strategy. Larger model parameters mean more VRAM

for GPUs, faster interconnects, and larger on-chip cache demands for NPUs. It also means higher energy consumption and cooling costs. In short, chasing a trillion-parameter model without a clear use case risks over-investing in infrastructure that sits idle between inference requests.

For many enterprises, the smarter move is to choose the right-size models, balancing accuracy with operational efficiency with precision. That's where methods like retrieval-augmented generation (RAG) and fine-tuning enter the picture, allowing enterprises to achieve targeted performance without scaling hardware budgets to breaking point.

RAG and fine-tuning the model

One of the most critical strategic decisions is whether to fine-tune the model so that knowledge is embedded

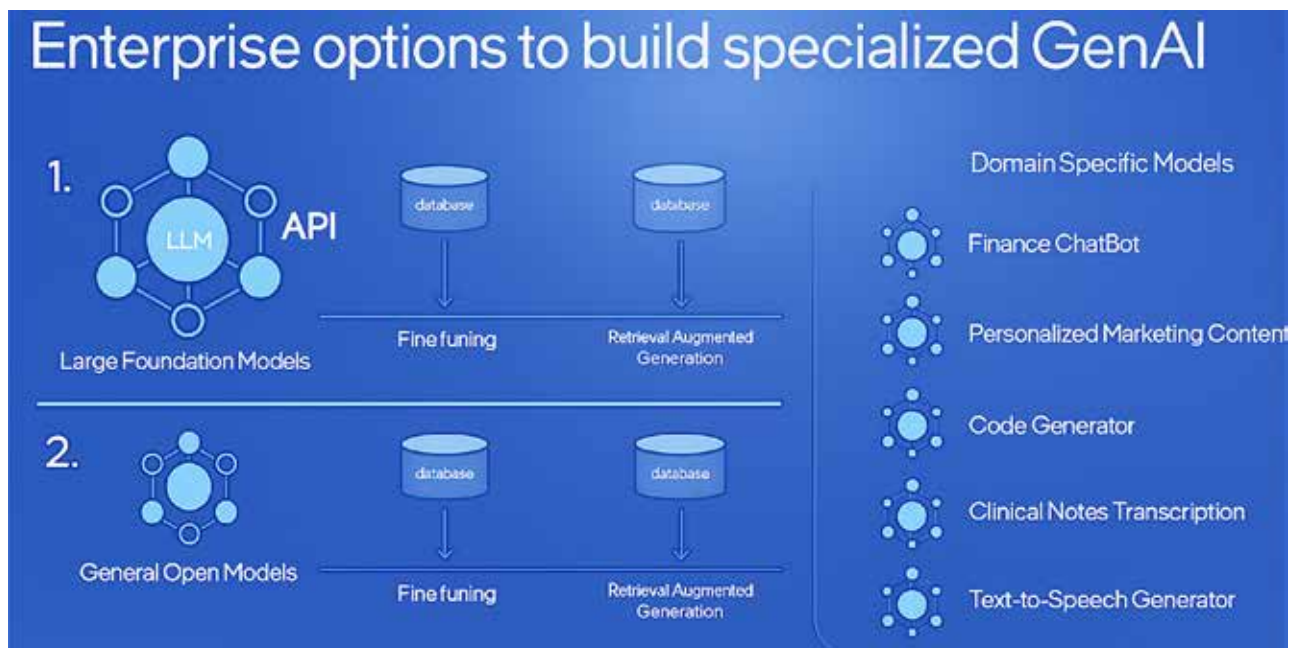
in its weights, or pair a smaller, generalist model with RAG for dynamic information.

Although ChatGPT-3 was trained on data spanning millennia, banks don't need historical breadth to answer loan or KYC queries. A focused 7 billion parameter Llama, fine-tuned on the bank's manuals, workflows, and local regulations, embeds domain-specific knowledge, delivering higher accuracy on industry-critical tasks than a far larger general-purpose model.

From the perspective of hardware compute requirements, this approach is dramatically more efficient: a 7 billion LLM model at 16-bit precision consumes around 67GB of RAM to load, whereas training a 200 billion parameter would demand thousands of high-end GPUs running non-stop for several months, with enormous power, cooling, and engineering overhead.

Model parameters dictate RAM requirements directly:

- **3 billion parameters** → ~30GB RAM load
- **7 billion parameters** → ~67GB RAM load
- **13 billion parameters** → well over 120GB RAM load



RAG and fine-tuning